educación IT
CODE YOUR FUTURE

Bootcamp DevOps Engineer

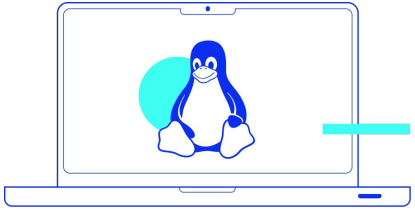
Fase 1 - SysAdmin

Módulo 3

Arranque, servicios y apagado

Mecanismos de arranque

Los 2 mecanismos principales de arranque en Linux son **`SysVinit`** y **`systemD`**, describiremos a estos dos con detalle a continuación.



SysVinit

Muchos sistemas operativos UNIX usan este mecanismo, que fue por primera vez implementado en UNIX System III y posteriormente en UNIX System V.

Proceso init

El primer proceso se llama **`init`**, que arranca luego de la carga del kernel. Al ser el primer proceso en ejecutarse toma el **PID 1**. El `init` es un *demonio* (es decir, corre en segundo plano y sin una terminal que lo controle), el cual es el padre de todos los procesos iniciados durante el uso del sistema.

Al iniciar el sistema tendrá un nivel de ejecución predeterminado *(runlevel)*, en el que se definirán los distintos procesos a iniciar. Entre los **niveles de ejecución** se encuentran:

- La configuración principal del `init` es el archivo ***/etc/inittab*** en el que se define cuál es el **nivel de ejecución predeterminado.**
- Luego de leer la configuración, el `init` cargará los *scripts* que se encuentran dentro de ***/etc/rc2.d*** (en caso de que el *runlevel* fuera el 2) o ***/etc/rc3.d*** (en caso de que el *runlevel* fuera el 3).

Niveles de Ejecución *(Runlevel)*

Los niveles de ejecución en **SystemV** describen ciertos estados del equipo, que se caracterizan por ejecutar ciertos procesos. En general, existen **8 niveles de ejecución,** que van del **0 al 6** y **S o s**, siendo estos últimos alias del mismo nivel de ejecución. De estos ocho niveles, tres son considerados reservados, y son los siguientes:

Nivel 0	Apagado del sistema. Inicia el proceso normal de apagado.
Nivel 1,s,S	Modo usuario único, se utiliza para la reparación del sistema, ya que iniciará solo los procesos necesarios para el arranque del sistema. Las tres opciones 1, s y S significan lo mismo.
Nivel 6	Reinicio del sistema. Inicia la secuencia de reinicio.

Dependiendo de la distribución:

Nivel 2	Nivel predeterminado, multiusuario con o sin entorno gráfico (Debian, Ubuntu).
Nivel 3	Nivel predeterminado, multiusuario sin entorno gráfico (Red Hat, Slackware, CentOS).
Nivel 4	Nivel multiusuario o reservado.
Nivel 5	Nivel multiusuario con entorno gráfico (Red Hat).

Single User Mode (Usuario único)

Este modo se utiliza para el **mantenimiento del sistema.** En este modo las conexiones remotas, red y la mayor parte de los servicios se encuentran desactivados. Generalmente se utiliza para corregir problemas del sistema de archivos, y que el sistema no puede resolver automáticamente.

Podemos acceder a este modo porque el sistema así lo pide o especificando por la línea de comandos del `GRUB`.

Línea de ejemplo:

```
kernel /vmlinuz-2.6.27.21-170.2.56.fc10.i686
ro root=/dev/hda1 rhgb quiet
```

Para **modo mantenimiento** se agrega al final de la línea un número **1** o la palabra **`single`** como parámetros.

Ejemplo con single:

```
linux /boot/vmlinuz-3.13.0-29-generic
root=/dev/sda1t single
```

Para **cambiar a modo single desde otro nivel** se utiliza **`telinit 1`** o **`init 1`**. Esta opción no es la más recomendada, porque no genera ningún tipo de aviso a los usuarios que se encuentran conectados. Más adelante se detalla cómo hacerlo.

En el modo mantenimiento no vamos a tener servicios de red ni uso de la mayoría de los servicios.

Estructura de los runlevels

Los directorios que veremos a continuación, por si solos, no cumplen función; es por eso que cumplen con una lógica que fue definida desde que se creó Unix System V, donde se especificó cómo tenía que estar situados cada demonio de arranque. Cuando un sistema Linux arranca, éste inicia una serie de *scripts* que se encuentran en **/etc**. Luego llama al nivel que corresponda para inicializar o detener los demonios requeridos.

La técnica de inicialización difiere entre algunas distribuciones de Linux, pero todas ellas tienen que cumplir con la **Linux Standard Base** para poder tener un orden genérico.

Ejemplo:

```
$ ls -la /etc/rc*
lrwxrwxrwx  1 root root    7 Apr 29 16:38 /etc/rc -> rc.d/rc
lrwxrwxrwx  1 root root   10 Apr 29 18:33 /etc/rc0.d -> rc.d/rc0.d
lrwxrwxrwx  1 root root   10 Apr 29 18:33 /etc/rc1.d -> rc.d/rc1.d
lrwxrwxrwx  1 root root   10 Apr 29 18:33 /etc/rc2.d -> rc.d/rc2.d
lrwxrwxrwx  1 root root   10 Apr 29 18:33 /etc/rc3.d -> rc.d/rc3.d
lrwxrwxrwx  1 root root   10 Apr 29 18:33 /etc/rc4.d -> rc.d/rc4.d
lrwxrwxrwx  1 root root   10 Apr 29 18:33 /etc/rc5.d -> rc.d/rc5.d
lrwxrwxrwx  1 root root   10 Apr 29 18:33 /etc/rc6.d -> rc.d/rc6.d
lrwxrwxrwx  1 root root   13 Apr 29 16:38 /etc/rc.local -> rc.d/rc.local
lrwxrwxrwx  1 root root   15 Apr 29 16:38 /etc/rc.sysinit -> rc.d/rc.sysinit

/etc/rc.d:
total 136
```

...

...

```
drwxr-xr-x  10 root root  4096 Apr 29 16:38 .
drwxr-xr-x 133 root root 12288 Aug 29 21:47 ..
drwxr-xr-x   2 root root  4096 Aug 28 20:18 init.d
-rwxr-xr-x   1 root root  2255 Jul  4  2009 rc
drwxr-xr-x   2 root root  4096 Aug 28 08:53 rc0.d
drwxr-xr-x   2 root root  4096 Aug 28 08:53 rc1.d
drwxr-xr-x   2 root root  4096 Aug 28 20:18 rc2.d
drwxr-xr-x   2 root root  4096 Aug 28 20:18 rc3.d
drwxr-xr-x   2 root root  4096 Aug 28 08:53 rc4.d
drwxr-xr-x   2 root root  4096 Aug 28 20:18 rc5.d
drwxr-xr-x   2 root root  4096 Aug 28 08:53 rc6.d
-rwxr-xr-x   1 root root   220 Jul  4  2009 rc.local
-rwxr-xr-x   1 root root 27239 Jul  4  2009 rc.sysinit
```

/etc/init.d

En este directorio se encuentran todos los *scripts* encargados de levantar cada uno de los servicios del sistema.

Algunos de los servicios que podemos encontrar en el **`init.d`** son los referentes a:

`httpd`	Servidor Web Apache
`smb`	Servidor Samba
`postfix`	Servidor de correo
`dhcpd`	Servidor DHCP
`named`	Servidor DNS
`mysqld`	Manejador de Base de Datos M

Los nombres pueden cambiar de acuerdo a la distribución utilizada:

```
# ls -l  /etc/rc.d/init.d/

-rwxr-xr-x 1 root root  2974 jun 23  10:18  dhcpd
-rwxr-xr-x 1 root root  3099 feb 25  2008   httpd
-rwxr-xr-x 1 root root  4239 mar  3  2008   mysqld
-rwxr-xr-- 1 root root  6154 ago  6  05:05  named
-rwxr-xr-x 1 root root  1745 sep 18  10:26  smb
-rwxr-xr-x 1 root root  4112 mar 29  2008   postfix
(...salida cortada...)
```

/etc/rc.sysinit o /etc/init.d/rcS

En sistemas basados en Red Hat, se hace uso del *script* **`rc.sysinit`** para la inicialización.

La secuencia de comandos **`rcS`** de Debian hace el mismo trabajo mediante la ejecución de varios pequeños *scripts* colocados en dos directorios diferentes.

En cada caso, el *script* se pone en marcha por **`init`** en el arranque. Este se ocupa de algunas tareas esenciales para preparar el sistema para su uso, tales como montar sistemas de ficheros.

Algunas de las tareas son:

- Configuración de reloj del sistema.
- Configuración de los parámetros del Kernel.
- Levantamiento de dispositivos RAID y LVM.
- Activación y actualización de cuotas en disco.
- Activación de la partición SWAP.

/etc/rc.local

Este archivo es un *script* que se llama después de todos los demás *scripts* de `init` (después de que todos los demonios del sistema han iniciado).

Contiene las **personalizaciones locales** que afectan el inicio del sistema y proporciona una alternativa para modificar los *scripts* de inicio de otros. Muchos administradores prefieren evitar cambiar `rc.sysinit`, porque esos cambios se perderán durante un sistema de actualización.

El contenido de `rc.local` no se pierde en una actualización.

Ejemplo:

```
# cat /etc/rc.local
#!/bin/sh
#
# This script will be executed *after* all the
other init scripts.
# You can put your own initialization stuff in here
if you don't
# want to do the full Sys V style init stuff.
touch /var/lock/subsys/local
```

/etc/rc

Este archivo es un *script* que se usa para **cambiar entre los niveles de ejecución.**

No se usa en Debian.

```
# ls -la /etc/rc
lrwxrwxrwx 1 root root 7 Apr 29 16:38 /etc/rc → rc.d/rc
```

El manejo de los servicio y el directorio /etc/init.d

La tarea de **iniciar y detener servicios del sistema** (también conocidos como *demonios*, cuya intención es ser ejecutados en segundo plano, como por ejemplo, un servidor web) está a cargo de archivos y enlaces simbólicos en **/etc/init.d** y por una serie de directorios de nivel de ejecución específicos llamados desde **/etc/rc0.d** a **/etc/rc6.d**.

En este directorio tendremos una serie de *scripts* que nos proveerán de diversas funcionalidades para cada servicio.

Son *scripts* que llevan argumentos que dan un comportamiento a éstos, ya sean los más básicos, como detener o iniciar un servicio.

Al ejecutar el `httpd` sin parámetros:

```
./httpd
Usage: httpd
{start|stop|restart|condrestart|reload|status|fulls
tatus|graceful|help|configtest}
```

Detener el servicio de `httpd`:

```
# ./httpd stop
Stopping httpd:                                [  OK  ]
```

Si **agregamos un nuevo servicio** por medio de un paquete **`rpm`** o **`dpkg`**, éste creará los *scripts* correspondientes sin tener que hacer nada. En otros casos tendríamos que agregarlos en el archivo ***rc.local*** o crear nuestros propios *scripts* de control.

Es importante tener en cuenta que estos archivos son simplemente ***shell scripts*** que contienen los parámetros usados normalmente para **administrar los demonios.**

No todos los demonios de Linux reconocen los argumentos detener, iniciar ,etc por la línea de comandos. Pero los *scripts* en **/etc/init.d** están estandarizados para que podamos ejecutarlos con los parámetros usados normalmente.

Los directorios /etc/rc0.d a /etc/rc6.d

Los *scripts* de inicio en **/etc/init.d** no son ejecutados directamente por el proceso `init`. En su lugar, cada uno de los directorios **/etc/rc0.d** a **/etc/rc6.d** contienen enlaces simbólicos que apuntan a los *scripts* del directorio **/etc/init.d**. Se usan enlaces simbólicos en lugar de archivos regulares para evitar trabajo innecesario en caso de querer cambiar un *script.*

Cuando el proceso de inicio entra en ejecución en un nivel **N**, se examinan todos los enlaces en el directorio **rcN.d** asociados. A estos enlaces se les dan nombres especiales en las formas de **`KNNname`** y **`SNNname`**. Vamos a explicarlo:

Prefijos K y S

Estas letras corresponden a matar ***(Kill)*** e iniciar ***(Start)*** respectivamente.

Cada nivel de ejecución define un estado en el que determinados servicios se están ejecutando y en otros no lo están.

El prefijo **S** se utiliza para marcar los archivos de todos los servicios que se van a ejecutar en determinado runlevel.

El prefijo **K** se utiliza para todos los demás servicios que no deben estar en ejecución en determinado runlevel.

Secuencia NN

Número de orden. Esta parte del nombre del enlace es un **entero de dos dígitos** (con un cero a la izquierda, si es necesario). En él, se especifica **el orden relativo de los servicios a iniciar o detener**. El número más bajo representa la primera secuencia de comandos ejecutados por `init`, y el mayor número es el último. No hay reglas estandarizadas para la elección de estos números, pero es importante que cuando añadamos un nuevo servicio, nos aseguremos que se inicie después o antes de los servicios que este necesite o dependa.

Si dos servicios poseen el mismo número, posiblemente se ejecuten por orden alfabético.

name (Nombre)

El nombre que se usa para el servicio tiene relación con lo que hace. Si el servicio se llama *networks*, éste tratará sobre algo de redes, si el servicio se llama *sshd*, tiene relación con el tipo de conexión segura, *secure shell.*

Cuando `init` tiene que iniciar un *runlevel*, usará el directorio correspondiente a su nivel (por ejemplo, /etc/rc2.d) y empezará a ejecutar los servicios que estén detallados.

El **orden** establecido de ejecución estará representado por el número que está a continuación de las letras **K** o **S**.

- La **K** es indicio de que ese *script* se llamará solamente para **apagar algo.**
- La **S** es indicio de que será utilizado para **iniciarlo** (Ej.: **`S10network`** luego **`S11auditd`**, **`S12restorecond`**, **`S12syslog`**).

Configurar el nivel de ejecución predeterminado

Para seleccionar el nivel predeterminado de arranque, hay que editar el archivo **/etc/inittab.**

Ejemplo de contenido. Definir el *runlevel* 5 como predeterminado.

```
id:5:initdefault:
```

Determinar el runlevel del sistema

Para verificar en qué *runlevel* nos encontramos, se utiliza el comando **`runlevel`**, el cual **mostrará el *runlevel* anterior y el actual.**

Si no hubo un nivel anterior, mostrará una N:

```
# runlevel
N 2
```

Cambiar el runlevel del sistema

Para cambiar de un *runlevel* al otro se utiliza el comando `init` o `telinit` (es un link simbólico).

Para reiniciar sin aviso

```
# init 6
INIT: Switching to runlevel: 6
INIT: sending processes the TERM signal
Stopping Postfix Mail Transport Agent: postfix
(...salida cortada...)
```

Para apagar sin aviso

```
# init 0
INIT: Switching to runlevel: 0
Stopping Postfix Mail Transport Agent: postfix
(...salida cortada...)
```

Comando chkconfig

Este comando es útil para **activar o desactivar los servicios que se aplican durante el arranque del equipo,** así como también **conocer el estado de los servicios** que se están ejecutando.

Este comando se usa en distribuciones de estilo Red Hat.

Para **conocer el estado de los procesos activos**, podemos ejecutar el siguiente comando:

```
# chkconfig -list
```

Para conocer el **estado de `httpd` en cada runlevel:**

```
# chkconfig --list httpd
httpd           0:desactivado   1:desactivado
2:desactivado   3:desactivado 4:desactivado
5:desactivado   6:desactivado
```

Para **deshabilitar** el servicio **apache2**:

```
# chkconfig httpd off
```

Para **habilitar** el servicio **apache2**:

```
# chkconfig httpd on
```

Para **activar el httpd** en los niveles 3 y 5:

```
# chkconfig --level 35 httpd on
```

Para **desactivar el httpd** en los niveles 3 y 5:

```
# chkconfig --level 35 httpd off
```

Para **desactivar `httpd` a mano** en los niveles 3 y 5:

```
# rm /etc/rc[35].d/S11httpd
```

Para **activar `httpd` a mano** en los niveles 3 y 5:

```
# ln -s /etc/init.d/httpd /etc/rc3.d/S11httpd
# ln -s /etc/init.d/httpd /etc/rc5.d/S11httpd
```

Comando update-rc.d

Este comando pertenece a las distribuciones basadas en **Debian** y es **análogo a `chkconfig`.**

Para **desactivar** el servicio **apache2**:

```
# update-rc.d -f apache2 remove
```

Para **activar** el servicio **apache2** en los niveles predeterminados:

```
# update-rc.d apache2 defaults
```

Levantando, deteniendo y reiniciando servicios

Para iniciar, detener, o reiniciar un servicio se puede poner la ruta completa del ejecutable en **`init.d`** seguido del parámetro.

```
# /etc/init.d/nombreDelServicio
{start|stop|status|restart|reload}
```

Supongamos que tenemos ya instalado y configurado un servidor web Apache y lo único que falta es iniciar el servicio. Para ello, solo bastará teclear lo siguiente:

```
# /etc/init.d/httpd start
```

Para **detener este servicio,** solo debemos cambiar la palabra `start` por **`stop`**:

```
# /etc/init.d/httpd stop
```

Para **reiniciar** el servicio:

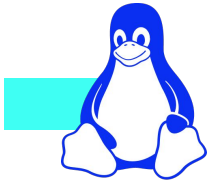
```
# /etc/init.d/httpd restart
```

Otra manera es mediante el comando **`service`**:

```
# service httpd {start|stop|status|restart|reload}
```

Para **iniciar** el servicio `httpd`:

```
# service httpd start
```



Upstart

Es un gestor de servicios alternativo a SysVinit. Se basa en la **detección de eventos.**

Los servicios se llaman *jobs* y en lugar de usar *scripts* de *shell*, poseen archivos de configuración con la extensión **.conf** dentro del directorio **/etc/init**.

Este mecanismo fue creado por Ubuntu, pero el proyecto está actualmente descontinuado.

Uso del comando wall

El comando **`wall`** se puede utilizar para **enviar un mensaje a todas las terminales** antes de cambiar de nivel de ejecución, apagar o reiniciar el sistema.

```
# wall "Pasaremos a modo monousuario para realizar
mantenimiento del sistema. Por favor, desconectarse."
```

Es posible **restringir el mensaje a un grupo** determinado del sistema. En este ejemplo, solamente los usuarios del grupo `desarrolladores` verán el mensaje:

```
# wall -g desarrolladores "Pasaremos a modo monousuario para
realizar mantenimiento del sistema. Por favor, desconectarse."
```

Uso del comando shutdown

Con el comando **shutdown**, podremos planear nuestra tarea de apagado o reinicio de sistema.

Sintaxis

```
# shutdown [opciones] tiempo [mensaje de alerta]
```

Opciones:

-f	Inicio rápido: omite el chequeo del disco al reiniciar.
-h	Apaga el equipo después de detener todo.
-k	Mensaje de alerta sin realizar el evento de apagado o de reiniciado.
-r	Reinicia después de detener todo.
-F	Fuerza al chequeo de discos después del reinicio.
-c	Cancelar `shutdown`.

Reiniciar el equipo en 60 segundos y **enviar aviso.**

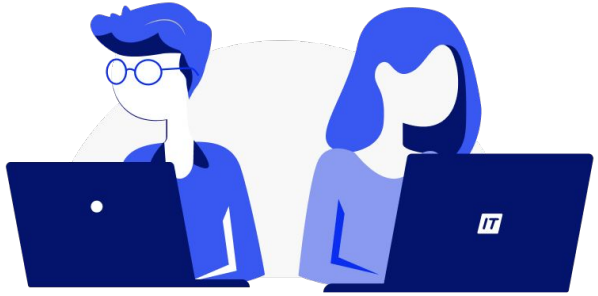
```
# shutdown -r 60 “salir del sistema”
```

Reiniciar el equipo ya mismo (ojo, usarlo si no hay nadie conectado):

```
# shutdown -r now
```

El botón de Power

El *hardware* actual en su gran mayoría soporta el **apagado de un equipo mediante el botón de *Power.*** Esto se puede lograr gracias al paquete **`acpid`** que proporciona el servicio del mismo nombre.



educación IT
CODE YOUR FUTURE

¡Sigamos trabajando!